MCU – SoC – FPGA

Đánh đổi giữa hiệu năng, chi phí và độ linh hoạt

GVHD: ...
Nhóm: ... (điền họ tên – MSSV)

Môn CO3053 Hệ thống nhúng · HK261
Trường Đại học Bách khoa – ĐHQG-HCM

Tháng 11/2026

VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY

Câu hỏi của bài

Khi nào nên chọn **MCU**, **FPGA** hay **SoC**?

- Xét theo 3 tiêu chí: **hiệu năng**, **chi phí**, **độ linh hoạt**
- *Không* nhằm chứng minh nền tảng nào "tốt nhất", vì đó là mục tiêu thiên vị mà pe2 cảnh báo
- Mỗi con số trên slide có một nhãn cho biết nguồn gốc:
  [đo] [mô phỏng] [ước lượng công cụ] [tính] [nguồn]

Lộ trình

1. **MCU**: đặc tính, điểm mạnh, điểm yếu, ứng dụng
2. **FPGA**: đặc tính, điểm mạnh, điểm yếu, ứng dụng
3. **SoC**: đặc tính, điểm mạnh, điểm yếu, ứng dụng
4. **So sánh**: số liệu tài liệu, thí nghiệm của nhóm, ma trận quyết định
5. Kết luận

Mỗi nền tảng được xét riêng trước, **theo cùng một khung**: là gì → mạnh ở đâu → yếu ở đâu → thắng ở lĩnh vực nào.

Sau đó mới đặt ba nền tảng **cạnh nhau** trên cùng một trục số liệu và cùng một bài toán FIR.

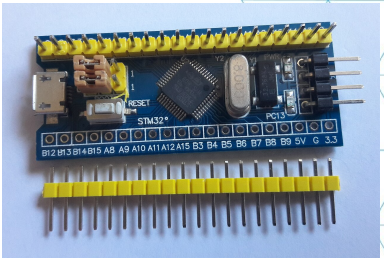
1

MCU

Vi điều khiển: nhỏ, rẻ, tiết kiệm điện, đúng giờ

MCU là gì?

- Một chip gồm **lõi CPU nhỏ** (ARM Cortex-M), **flash**, **SRAM** và **ngoại vi**: timer, ADC, PWM, UART, SPI, I^2C
- Chạy **bare-metal** hoặc **RTOS**, lập trình bằng C, truy cập thanh ghi trực tiếp (Lecture 2)
- Thực thi **tuần tự**: mỗi chu kỳ làm một việc
- **Không có MMU**, nên không chạy được Linux
- Board nhóm dùng: Blue Pill STM32F103C6, Cortex-M3 72 MHz



Popolon, CC BY-SA 4.0

Board Blue Pill (họ STM32F103)

MCU: điểm mạnh và điểm yếu

Điểm mạnh

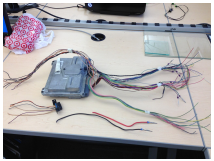
- **Rẻ**: H743 £9–12/chip; dòng F103 chỉ vài USD [nguồn]
- **Cực tiết kiệm khi nghỉ**: STM32U5 300 nA, H743 2,43 µA ở Standby [nguồn]; pin CR2032 dùng hơn 10 năm (TI)
- **Đúng giờ**: ngắt bare-metal 1–3 µs [nguồn: AC6]; nhóm đo: 264–347 ns [đo]
- **Khởi động trong vài ms**, chạy được ngay khi cấp điện
- **Có chứng nhận an toàn**: AURIX đạt ASIL-D, AEC-Q100 Grade 0 chịu tới 150 °C
- **Ra thị trường nhanh** với chức năng đơn giản, cố định (Lecture 2)

Điểm yếu

- **Hiệu năng thấp**: CoreMark 608–2 424 [nguồn]
- **Không đủ sức cho xử lý song song**: H743 chỉ ≈0,48 G MAC/s, bằng ≈29% nhu cầu của bài lọc video [tính]
- **Không có Linux**: phải tự viết TCP/IP, file system, driver camera
- **Worst-case do phần mềm quyết định**: thêm một ngắt ưu tiên cao 20 µs là độ trễ lên 21,9 µs [đo]
- M3 **không có SIMD**: bản nhanh nhất vẫn mất 216,3 chu kỳ/mẫu, CMSIS-DSP không nhanh hơn C [đo]

MCU thắng ở đâu: ECU ô tô

- Xe phổ thông có 25–35 MCU, xe sang 60–70
- AURIX TC3xx: 4/6 lõi chạy **lockstep**, nhắm tới **ASIL-D** (ISO 26262), dùng cho powertrain và inverter xe điện
- AEC-Q100 Grade 0: chạy được từ −**40 đến 150 °C**; NXP S32K14xW cam kết cung ứng 15 năm
- Khởi động vài ms, ngắt dưới 1 µs, giá vài USD



Snewkirk7953, CC BY-SA 3.0

Vì sao không chọn SoC: Linux khởi động lâu, độ trễ không xác định

Vì sao không chọn FPGA: đắt; cấu hình nằm trong SRAM nên dễ bị lỗi bức xạ (SEU); khó chứng nhận cho hàng triệu xe

Ưu thế dừng ở: ADAS và giải trí dùng SoC; xu hướng domain controller gom nhiều ECU thành ít bộ hơn

Cũng chiếm ưu thế: điều khiển động cơ FOC (dưới 21 µs/50 µs), cảm biến chạy pin (300 nA, hơn 10 năm trên pin CR2032), đồ gia dụng

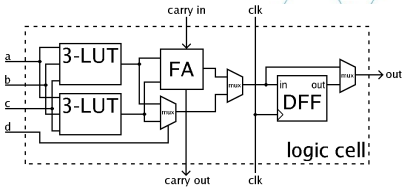
2

FPGA

Mạch logic khả trình: song song thật, độ trễ cố định

FPGA là gì?

- Một mảng hàng nghìn **logic cell** (LUT + flip-flop), cùng khối **DSP** (nhân-cộng) và **BRAM**, nối với nhau tùy ý
- Không chạy chương trình: thuật toán được **"trải ra không gian"**, mỗi phép tính có một mạch riêng chạy đồng thời
- Mô tả bằng **HDL** (Verilog, VHDL); cấu hình nằm trong SRAM nên **nạp lại được**
- Board nhóm dùng: Arty Z7-20, phần PL của XC7Z020 (220 DSP48E1, 53 200 LUT); FIR của nhóm chạy **thuần LUT, 0 DSP**



Petter.kallstrom, Public domain

Một logic cell: LUT, bộ cộng, flip-flop



S03311251, CC BY-SA 4.0

FPGA: điểm mạnh và điểm yếu

Điểm mạnh

- **Song song thật**: 220–240 DSP ≈44–48 G MAC/s ở 200 MHz [tính]; XC7A100T công bố tối đa 264 G [nguồn]
- **1 mẫu/chu kỳ**: FIR của nhóm đạt 1,00 mẫu/chu kỳ [đo]
- **Độ trễ cố định, cỡ ns**: tick-to-trade 13,9 ns [nguồn]; pipeline của nhóm 7 chu kỳ = 56 ns
- **Tiết kiệm năng lượng cho mỗi phép tính**: FIR trên FPGA tốn ít hơn CPU ≈95–190× [tính, Arucu & Iliev]
- **I/O nối thẳng vào chân chip**, và **nạp lại được** khi chuẩn thay đổi

Điểm yếu

- **Đắt**: XC7A100T (dòng Artix-7) €143,90/chip [nguồn]
- **Khó lập trình**: mọi phép toán phải tự mô tả ở mức phần cứng (Arucu & Iliev)
- **Linh hoạt giảm**: thêm FPGA hạ độ linh hoạt từ “Cao” xuống “Trung bình” (Lecture 3)
- **Không có OS**: network stack hay giao diện người dùng đều phải tự làm
- **Không thắng ở bài nhỏ**: một lõi CPU desktop lọc FIR 16-tap được ≈140 MSPS [đo, x86 tham chiếu]

FPGA thắng ở đâu: dòng dữ liệu song song và tạo mẫu chip

"Tính toán song song" nói chung thì GPU phổ biến hơn. FPGA thắng khi dữ liệu **chảy liên tục từ ADC**, cần **độ trễ cố định**, và chuẩn còn thay đổi hoặc số lượng không đủ lớn để làm ASIC.

- **Siêu âm y tế:** beamforming 128–1024 kênh, 4,83 GS/s trên một FPGA; nối thẳng với front-end analog
- **Trạm 5G (radio unit):** low-PHY và DFE trên RFSoC hoặc Agilex; chuẩn O-RAN còn đổi nên cần nạp lại được
- **Tạo mẫu ASIC:** HAPS ghép nhiều FPGA chạy RTL tới 1,6 tỉ cổng trước khi đúc chip
- **HFT:** tick-to-trade 13,9 ns



Erereony, CC BY 4.0

Vì sao không chọn MCU: thiếu ≈3,5× năng lực dù đạt hiệu suất 100%

Vì sao không chọn SoC: có thể đủ thông lượng nhờ SIMD/GPU, nhưng mất tính xác định và tốn điện

Ưu thế dừng ở: số lượng lớn và thuật toán ổn định thì ASIC thắng (ISP, modem trong điện thoại)

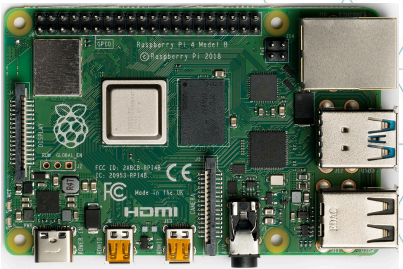
3

SoC

Hệ thống trên chip: CPU mạnh + Linux + khối tăng tốc

SoC là gì?

- Lõi **ứng dụng** (ARM Cortex-A) có **MMU** và **cache**, cùng GPU, bộ giải mã video và khối tăng tốc mạng, tất cả trên một chip
- Chạy **Linux**: driver, network stack, file system có sẵn (Lecture 2)
- Khối cứng chuyên dụng (xử lý gói tin, ISP, NPU, modem) làm phần nặng; CPU lo phần điều khiển
- Ví dụ nhóm dùng: Raspberry Pi 4, 4×Cortex-A72 1,8 GHz



Laserlicht, CC BY-SA 4.0

Raspberry Pi 4 Model B

SoC: điểm mạnh và điểm yếu

Điểm mạnh

- **CPU mạnh nhất trong ba nền tảng**: Pi 4 đạt CoreMark 38 200, gấp ≈16× H743 [nguồn]
- **Có Linux**: camera chạy ngay qua V4L2, mạng, file system, cập nhật từ xa
- **Tích hợp khối cứng**: router xử lý gói tin 37,5 Mpps; điện thoại có NPU, ISP, modem 5G [nguồn]
- **Board rẻ**: Pi 4 $35 khi ra mắt [nguồn]
- **Ra thị trường nhanh** với sản phẩm nhiều tính năng, có kết nối (Lecture 2)

Điểm yếu

- **Độ trễ không xác định**: PREEMPT_RT 30–80 µs, Linux thường vài ms [nguồn: AC6]
- **Tốn điện ngay cả khi rảnh**: Pi 4 ≈3 W cả board, ≈7 W lúc đỉnh [nguồn]
- **Khởi động lâu**, phần mềm phức tạp (kernel, driver, bảo mật)
- **Yếu với thời gian thực cứng** nếu không có bản vá real-time (Lecture 2)

SoC thắng ở đâu: router Wi-Fi và điện thoại

- **Router:** Qualcomm IPQ = CPU chạy Linux/OpenWrt + Wi-Fi + switch + **khối xử lý gói tin cứng** (37,5 Mpps) + mã hóa
- **Điện thoại:** Snapdragon 8 Elite = CPU + GPU + NPU + ISP + modem 5G, chip 3 nm
- Linux lo điều khiển (control plane), khối cứng lo dữ liệu (data plane)



Raimond Spekking, CC BY-SA 4.0



Raimond Spekking, CC BY-SA 4.0

Cũng chiếm ưu thế: camera IP, TV và set-top box, gateway IoT, giải trí trên xe

Vì sao không chọn MCU: không có MMU nên không chạy Linux; phải tự viết lại TCP/IP stack

Vì sao không chọn FPGA: viết cả network stack và giao diện bằng HDL quá đắt cho thiết bị tiêu dùng

Ưu thế dừng ở: điều khiển thời gian thực cứng, nên trong máy vẫn có thêm MCU nhỏ

Khi cần cả hai: phương án lai CPU + FPGA

- Bài toán cần **cả** hệ điều hành phong phú **lẫn** timing hoặc xử lý song song cứng (ADAS, điều khiển công nghiệp)
- Zynq UltraScale+ gom trên một chip: 4×A53 chạy Linux, 2×R5 lockstep chạy real-time, và fabric FPGA
- Lecture 3, Option #2: CPU lo tính toán mềm dẻo, FPGA giữ timing actuator kể cả khi CPU bận
- FIR trên SoC FPGA+ARM nhanh gấp 8,86× ARM đơn, giá/hiệu năng tốt nhất trong các cấu hình được thử [nguồn]



S03311251, CC BY-SA 4.0

Zynq UltraScale+: Arm + FPGA trên một chip

Cái giá: chi phí cao hơn và linh hoạt thấp hơn so với một CPU đơn

4

So sánh

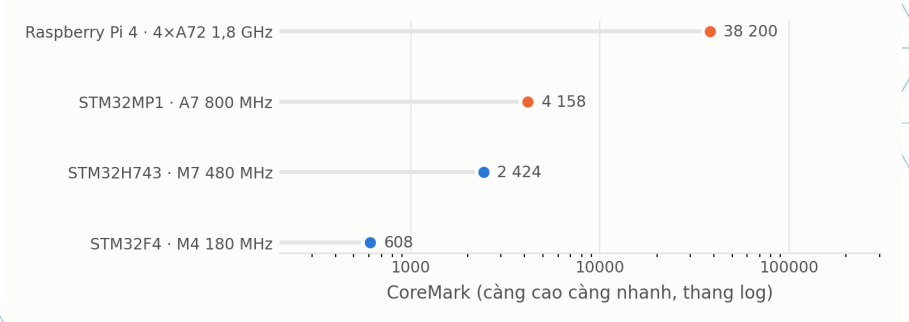
Đặt ba nền tảng cạnh nhau trên cùng một trục

Tổng quan đánh đổi

Tiêu chí	MCU	FPGA	SoC
CPU tuần tự (CoreMark)	Thấp: 608–2 424	Không áp dụng	**Cao: 38 200** (Pi 4)
Thông lượng song song	≈0,48 G MAC/s [tính]	≈**48 G MAC/s** [tính]	Trung bình (SIMD, GPU)
Độ trễ xấu nhất	1–3 µs	**cỡ ns**	30 µs – vài ms
Công suất khi nghỉ	**cỡ µW** (chip)	chưa có số	≈3 W (board)
Giá	**£9–12/chip**	€144/chip	$35/board
Lập trình	C, thanh ghi	HDL, khó nhất	**Linux, nhiều thư viện**
Linh hoạt / ra thị trường	Nhanh với bài đơn giản	Thấp nhất	Nhanh với bài nhiều tính năng

Nguồn: ST, OpenBenchmarking, AC6, STAC-T0, RS, Farnell, AnandTech, Lecture 2–3. Chữ đậm là nền tảng thắng ở tiêu chí đó. Các slide sau đi vào từng tiêu chí.

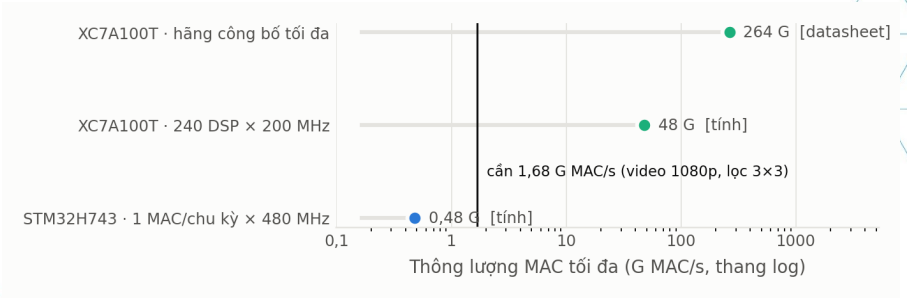
Hiệu năng CPU thuần: CoreMark



Dữ liệu: ST product overview; ST STM32H743; OpenBenchmarking (Pi 4, tổng 4 lõi). Nhóm vẽ lại.

SoC Pi 4 mạnh gấp ≈16× **MCU** H743 về CPU. CoreMark không áp dụng cho **FPGA**, vì FPGA không chạy chương trình tuần tự.

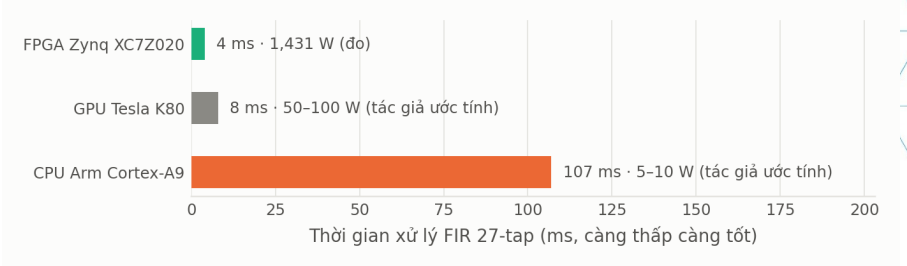
Thông lượng song song: bài lọc video 1080p



Dữ liệu: Lee & Seshia mục 8.1 (nhu cầu); datasheet XA Artix-7 (264 G). Giá trị [tính] theo giả thiết trong bài. Nhóm vẽ lại.

MCU chỉ đạt ≈29% nhu cầu dù hiệu suất 100%; **FPGA** chỉ dùng ≈3,5% năng lực [tính]

Bằng chứng đã công bố: FIR trên FPGA, GPU và CPU

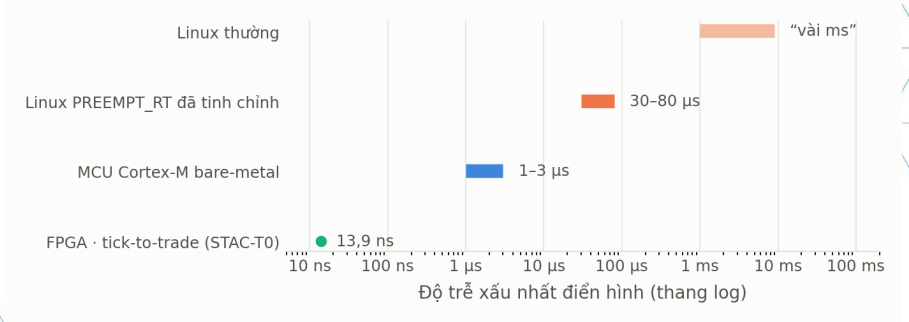


Dữ liệu: Arucu & Iliev, J. Low Power Electron. Appl. 15(3):40, 2025. Nhóm vẽ lại.

FPGA nhanh hơn Cortex-A9 ≈27× và tốn ít năng lượng hơn ≈95–190× [tính].

Công suất CPU và GPU là ước tính của tác giả; bài không có MCU.

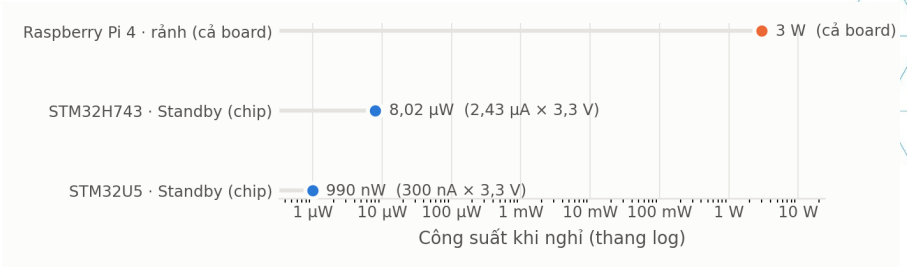
Độ trễ xấu nhất theo tài liệu



Dữ liệu: AC6 (MCU, Linux); A-Team Insight, STAC-T0 (FPGA). “Vài ms” không có số chính xác nên vẽ mờ. Nhóm vẽ lại.

Mỗi nền tảng cách nhau khoảng 1–2 bậc độ lớn. Các phép đo khác bản chất nhau, chỉ so bậc độ lớn.

Công suất khi nghỉ và giá



Dữ liệu: ST STM32H743; Arrow (STM32U5); TechFeed (Pi 4). Công suất MCU = dòng × 3,3 V [tính]. Nhóm vẽ lại.

Chênh ≈3 triệu lần: lý do **MCU** thống trị thiết bị chạy pin.
Giá: **MCU** £9–12/chip · **SoC** $35/board · **FPGA** €144/chip [nguồn; so chip với board]

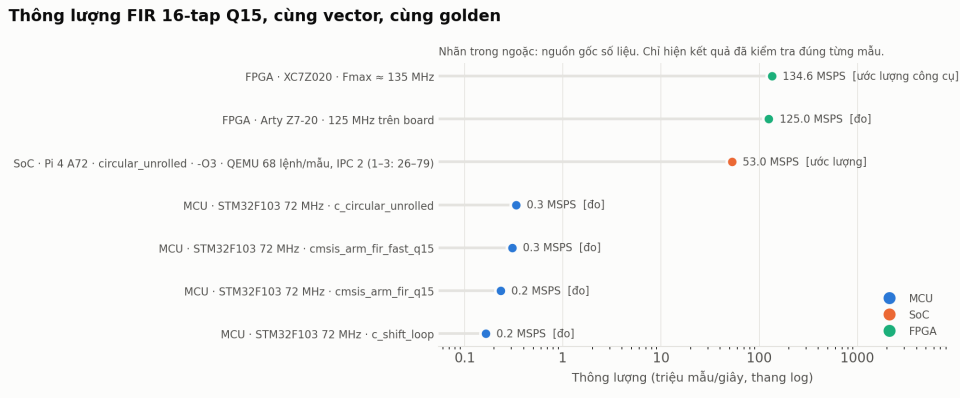
Thí nghiệm của nhóm: một bài toán, ba nền tảng

FIR 16-tap Q15, 1024 mẫu: bậc thang, giá trị biên, mẫu ép bão hòa, mẫu ngẫu nhiên
Mọi nền tảng phải khớp **từng mẫu** với golden model viết bằng Python

MCU	STM32F103, Cortex-M3	4 phiên bản (C, CMSIS-DSP), đếm chu kỳ bằng DWT; độ trễ ngắt trong 3 kịch bản
FPGA	Arty Z7-20 (XC7Z020 PL)	Pipeline 7 tầng thuần LUT (0 DSP); tự kiểm ngay trên chip ở 125 MHz; Vivado cho Fmax và công suất
SoC	Raspberry Pi 4 (A72)	Cùng mã C, `-O2` và `-O3` (NEON); `cyclictest`. Trước khi có board: QEMU kiểm tra đúng bit và đếm lệnh/mẫu

Cortex-M3 không có lệnh SIMD (SMLAD), nên CMSIS-DSP không được tăng tốc kiểu SIMD như trên M4

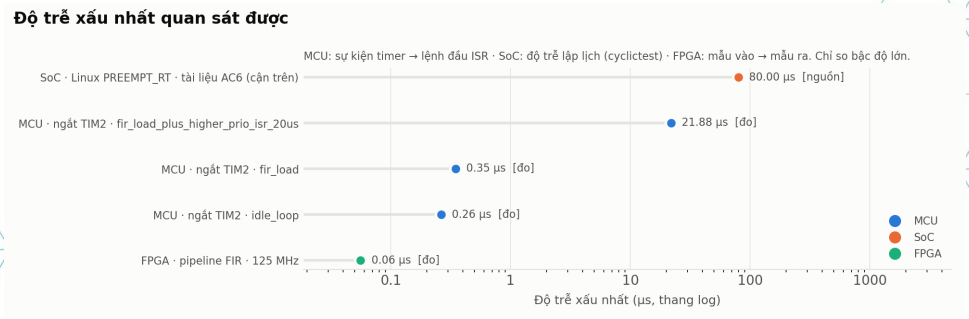
Kết quả: thông lượng



MCU: 0,33 MSPS [đo] (**FPGA** gấp 375×) **FPGA**: 1,00 mẫu/chu kỳ, 125,0 MSPS [đo]

SoC: 53,0 MSPS [ước lượng; IPC 1–3: 26–79]

Kết quả: độ trễ xấu nhất



MCU ngắt khi chạy FIR: 264–347 ns · thêm ngắt ưu tiên cao hơn 20 µs: 21,9 µs [đo]
FPGA pipeline: 7 chu kỳ = 56 ns [đo] **SoC** Linux thường: vài ms [nguồn: AC6] · PREEMPT_RT: 30–80 µs [nguồn: AC6]

Kiểm chứng chéo với tài liệu

Câu hỏi	Nhóm đo được	Tài liệu nói
MCU đạt được bao nhiêu so với “1 MAC/chu kỳ”?	7% (216,3 chu kỳ/mẫu)	PLAN giả định 100%; flash cần 2 wait state ở 72 MHz, dữ liệu `const` đọc từ flash
Độ trễ ngắt MCU bare-metal	264–347 ns	1–3 µs (AC6)
FPGA nhanh hơn MCU / CPU bao nhiêu lần?	375× (vs MCU); 2,4× (vs Pi 4, ước lượng)	≈27× (so với A9, Arucu & Iliev)
SoC chạy đúng trên ARMv8 (Cortex-A72)?	PASS (QEMU), 68 lệnh/mẫu	—
Tài nguyên FPGA cho FIR 16-tap (thuần LUT)	0/220 DSP · 1 672 LUT · 1 046 FF	Fmax ≈ 135 MHz, 0,124 W [ước lượng công cụ]

Quan sát phụ: một lõi CPU x86 lọc được ≈140 MSPS. FPGA không thắng nhờ tốc độ thô ở bài nhỏ, mà nhờ **thông lượng giữ nguyên khi tăng số tap**, **độ trễ cố định** và **công suất thấp**.

Tóm tắt: mỗi nền tảng thắng ở đâu

	Lĩnh vực đặc trưng	Yếu tố quyết định	Vì sao không chọn hai nền tảng kia
MCU	ECU ô tô, động cơ, cảm biến chạy pin	ASIL-D, 150 °C, dòng nghỉ cỡ nA, vài USD	SoC chậm khởi động và không xác định; FPGA đắt
FPGA	Siêu âm, 5G radio, tạo mẫu ASIC, HFT	Song song + I/O trực tiếp + tái cấu hình	MCU thiếu sức; SoC không xác định
SoC	Router Wi-Fi, điện thoại, camera IP	Linux + khối tăng tốc cứng	MCU không có MMU; FPGA tốn công HDL
Lai	ADAS, công nghiệp	OS + timing cứng	Một nền tảng đơn không đủ

Ma trận quyết định

Nếu bài toán…	Chọn	Vì
Chạy pin nhiều năm, tác vụ nhẹ	**MCU**	300 nA ở Standby
Deadline vài chục µs, cần chứng nhận an toàn	**MCU**	Ngắt dưới 1 µs; ASIL-D
Trên ≈0,5 G MAC/s liên tục, I/O tốc độ cao	**FPGA** / lai	1 mẫu/chu kỳ, ≈44–48 G MAC/s
Độ trễ cỡ ns	**FPGA**	13,9 ns
Cần Linux, camera, mạng; timing mềm	**SoC**	CoreMark gấp ≈16×; driver có sẵn
Cần cả OS lẫn timing cứng	Lai	Zynq, Lecture 3

Muốn timing và thông lượng cao hơn thì phải trả bằng **giá cao hơn** và **linh hoạt thấp hơn**

5
Kết luận

“Không có nền tảng tốt nhất,

chỉ có nền tảng **đủ** cho con số quyết định của bài toán,
với chi phí thấp nhất.”

— Nguyen Vo Hoang, Master’s student, NTU

Hỏi đáp

Phụ lục: chuẩn bị cho câu hỏi

- **“MCU đã tối ưu chưa?”** Có bản CMSIS-DSP. M3 không có SIMD, bản tốt nhất đạt 216,3 chu kỳ/mẫu.
- **“Jitter của FPGA bằng 0 thật à?”** Mô phỏng RTL không có thời gian thật. Trên chip, độ trễ cố định theo số chu kỳ, chỉ còn clock jitter cỡ ps.
- **“Sao so chip với board?”** Chỉ để so bậc độ lớn, và nhóm đã ghi rõ trên từng bảng.
- **“Công suất FPGA đo bằng gì?”** Không đo; đó là Vivado `report_power` (vector-less) [ước lượng công cụ].
- **“Sao không có NRE?”** Chưa có số công bố. Nhóm dùng thời gian thực tế viết bản C và bản HDL làm dẫn chứng định tính.
- **“Tăng lên 64 tap thì sao?”** MCU chậm đi khoảng 4×. FPGA vẫn 1 mẫu/chu kỳ, chỉ tốn thêm LUT/DSP và vài tầng pipeline.

Nguồn chính

- ST: STM32H7/F4/MP1 overview, STM32H743; OpenBenchmarking: Raspberry Pi 4
- Arucu & Iliev, J. Low Power Electron. Appl., 2025 (FIR trên FPGA, GPU, CPU)
- AC6: độ trễ Linux PREEMPT_RT; A-Team Insight: Exegy/AMD STAC-T0
- Embedded World 02/2026 (số MCU mỗi xe); Automotive World (AURIX TC3xx); Future Electronics (NXP S32K Grade 0)
- Macnica, CNX Software (Qualcomm IPQ); GSMArena (Snapdragon 8 Elite)
- Kou et al., 2023 (beamforming siêu âm trên FPGA); HPCwire (Fujitsu + Xilinx O-RAN); Intel (Agilex O-RU); Synopsys HAPS
- eeNews Europe (Zynq UltraScale+); Lecture 2, Lecture 3; Lee & Seshia mục 8.1

Danh sách đầy đủ kèm đường link nằm trong báo cáo (`report.pdf`).

Template slide dựa trên “HCMUT-Slide” của Nguyễn Quang Đức (Overleaf, CC BY 4.0).

Nguồn hình ảnh

Ảnh chụp: Wikimedia Commons, giữ nguyên, chỉ thu nhỏ. Biểu đồ ghi “Nhóm vẽ lại” được dựng từ số liệu đã công bố, không chép hình gốc.

- Core Learning Board module Arduino STM32 F103 C8T6.jpg — Popolon, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Core_Learning_Board_module_Arduino_STM32_F103_C8T6.jpg
- Raspberry Pi 4 Model B - Top.jpg — Laserlicht, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Raspberry_Pi_4_Model_B_-_Top.jpg
- Xilinx Artix7 XC7A100T FGG484 on PCB.jpg — S03311251, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Xilinx_Artix7_XC7A100T_FGG484_on_PCB.jpg
- FPGA cell example.png — Petter.kallstrom, Public domain. https://commons.wikimedia.org/wiki/File:FPGA_cell_example.png
- ECU and wire bundles.JPG — Snewkirk7953, CC BY-SA 3.0. https://commons.wikimedia.org/wiki/File:ECU_and_wire_bundles.JPG
- Intel FWIXP422BB on mainboard of UMTS Router Surf@home II, o2-8338.jpg — Raimond Spekking, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Intel_FWIXP422BB_on_mainboard_of_UMTS_Router_Surf@home_II,_o2-8338.jpg
- HTC Desire - main board - Qualcomm QSD8250-2.jpg — Raimond Spekking, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:HTC_Desire_-_main_board_-_Qualcomm_QSD8250-2.jpg
- Sonoscape Ultrasound Machine.jpg — Erereony, CC BY 4.0.
  https://commons.wikimedia.org/wiki/File:Sonoscape_Ultrasound_Machine.jpg
- Xilinx ZynqUltrascale XCZU4CG on PCB.jpg — S03311251, CC BY-SA 4.0.
  https://commons.wikimedia.org/wiki/File:Xilinx_ZynqUltrascale_XCZU4CG_on_PCB.jpg